

Ayush's OraVision AWR Pro

Oracle AWR Performance Dashboard

TECHNICAL ARCHITECTURE DOCUMENT

Version: 1.0

Date: April 2026

Classification: Internal / Technical

Audience: Engineering, QA, Architecture Review

Table of Contents

1. Executive Overview	3
2. System Architecture	3
3. Technology Stack	4
4. Repository & File Structure	5
5. End-to-End Request Flow	6
6. AWR Parsing Engine	7
7. Key Algorithms & Scoring	8
8. API Reference	10
9. Frontend Architecture	11
10. Deployment Options	12
11. Security Considerations	13
12. Supported Oracle Versions	13
13. Dependency Manifest	14
14. Glossary	15

1. Executive Overview

OraVision AWR Pro is a self-hosted, web-based analytics dashboard purpose-built for Oracle Database performance engineers. It accepts Oracle Automatic Workload Repository (AWR) HTML reports — the industry-standard diagnostic exports produced by `awrrpt.sql` — and transforms them into interactive visualisations, comparative analyses, root-cause assessments, and prioritised remediation recommendations, all without requiring any Oracle database connection at runtime.

The tool targets DBAs, performance consultants, and SRE teams who need to identify performance regressions between two snapshot periods, isolate CPU, I/O, or latch bottlenecks, and generate board-ready root-cause reports in seconds rather than hours of manual inspection.

1.1 Problem Statement

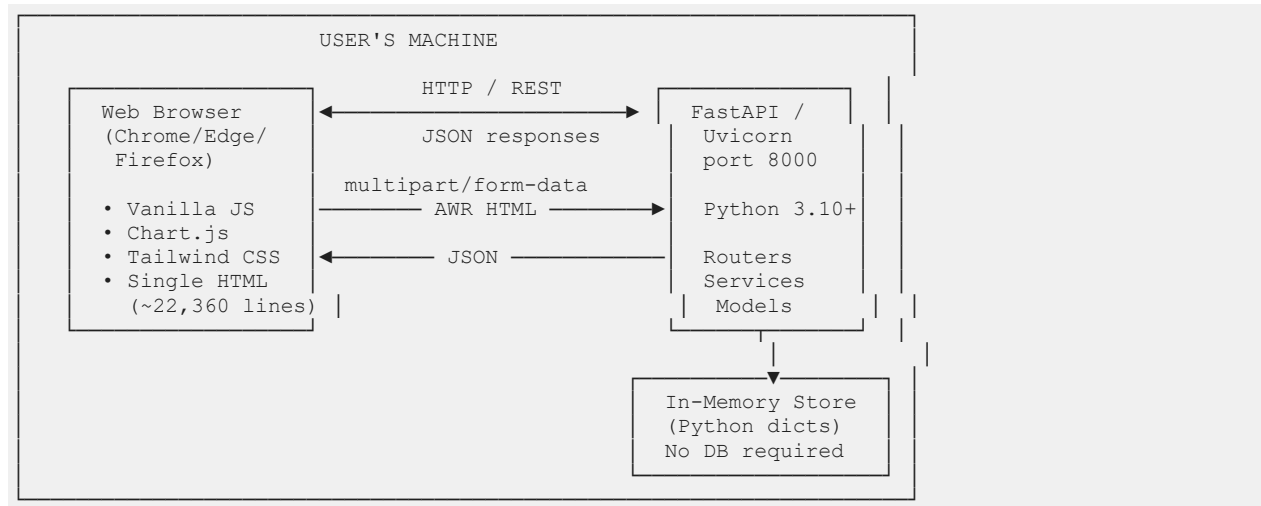
Oracle AWR HTML reports are extremely rich but are also hundreds of kilobytes of raw HTML tables. Reading them manually is error-prone and slow. Comparing two periods requires a DBA to open both files simultaneously, mentally reconcile dozens of metrics, and then write up findings by hand. OraVision AWR Pro automates all of this end-to-end.

1.2 Key Capabilities

- Upload and parse one or two Oracle AWR HTML reports (11g through 21c).
- Single-report mode: full health score, wait-event breakdown, top SQL, load-profile view.
- Comparison mode: delta analysis for every metric, SQL regression classification, bottleneck determination, workload-pattern detection.
- Health scoring engine: 0–100 weighted score with colour-coded risk bands.
- Root-cause analysis (RCA) engine: deterministic rule-based diagnosis with ranked recommendations.
- In-browser RCA report download (HTML format, no server round-trip).
- Zero-dependency client: users only need a modern web browser and Python 3.10+.
- Windows launcher (`start.bat`) for non-technical users — auto-installs, auto-opens browser.

2. System Architecture

OraVision AWR Pro follows a classic single-host client–server architecture. The server runs entirely on the user's local machine (or a shared jump server). There is no external database, no cloud back-end, and no data ever leaves the machine.



All parsed AWR data is held in Python dictionaries in process memory for the lifetime of the server session. There is no relational database, no file-system cache, and no persistent state between server restarts.

2.1 Architectural Pattern

- Pattern: Monolithic single-process server with REST API.
- Transport: HTTP/1.1 over TCP loopback (127.0.0.1:8000).
- Concurrency model: ASGI (Async Server Gateway Interface) via Uvicorn + asyncio. Upload parsing runs in a thread pool to avoid blocking the event loop.
- State management: in-process Python dictionaries keyed by a period identifier ('baseline' or 'comparison').
- Frontend rendering: client-side JavaScript — the server delivers one HTML file; all DOM manipulation, chart rendering, and diff computation happen in the browser.

3. Technology Stack

3.1 Backend

Component	Technology	Version	Purpose
Runtime	Python	3.10+	Application language
Web framework	FastAPI	0.111+	REST API, routing, dependency injection, OpenAPI docs
ASGI server	Uvicorn	0.29+	Production-grade ASGI server, handles HTTP, keep-alive
HTML parser	BeautifulSoup4	4.12+	Parses Oracle AWR HTML tables via lxml back-end
HTML back-end	lxml	5.0+	C-based XML/HTML parser (10–20× faster than html.parser)
Data models	Pydantic v2	2.0+	Runtime type validation, serialisation of AWR data structures
Templates	Jinja2	3.1+	Server-side rendering of the main index.html
Static files	Starlette	(bundled)	Serves CSS/JS assets from /static/
CORS	Starlette CORS	(bundled)	Allows browser fetch from same origin

3.2 Frontend

Component	Technology	Delivery	Purpose
Language	Vanilla JavaScript (ES2020+)	Inline in HTML	All UI logic, API calls, DOM updates
Styling	Tailwind CSS	CDN (jsDelivr)	Utility-first CSS, dark-mode theme
Charts	Chart.js 4.x	CDN (jsDelivr)	Bar, doughnut, and line charts
Icons	Heroicons (SVG inline)	Inline in HTML	UI icons — no icon font dependency
HTTP client	Fetch API (native browser)	Built-in	REST calls to FastAPI backend
Drag & Drop	Native HTML5 DragEvent	Built-in	AWR file upload interaction
File parsing	FileReader API	Built-in	Reads dropped file for upload
PDF export	window.print() CSS media query	Built-in	Browser-native print-to-PDF

3.3 Development & Distribution Tools

Tool	Purpose
start.bat	Windows launcher: Python discovery, port cleanup, pip install, browser open
_open_browser.py	Socket-polls port 8000, opens browser the moment server is ready
requirements.txt	Pinned dependency list for pip install
OraVision-AWR-Pro.zip	Single-file distribution artefact for internal sharing

4. Repository & File Structure

The project ships as a single directory tree:

```
awr-dashboard/
├── start.bat                ← Windows launcher (double-click to run)
├── _open_browser.py         ← Browser-open helper (auto-launched by start.bat)
├── README_tester.md        ← Non-technical user guide
├── backend/                ← All server-side code
│   ├── main.py             ← FastAPI application entry-point
│   ├── requirements.txt     ← Python dependencies
│   ├── routers/            ← API route handlers (one file per domain)
│   │   ├── upload.py       ← POST /api/upload/awr
│   │   ├── snapshots.py    ← GET /api/snapshots/
│   │   ├── compare.py      ← GET /api/compare/{...}
│   │   ├── sql_analysis.py ← GET /api/sql/top/{period}
│   │   ├── wait_events.py  ← GET /api/waits/{period}
│   │   ├── recommendations.py ← GET /api/recommendations/{period}
│   │   ├── intelligence.py ← GET /api/intelligence/{upload_id}
│   │   ├── rag.py          ← POST /api/rag/narrative, /api/rag/learn
│   │   └── ai_rca.py       ← POST /api/ai-rca/analyze
│   ├── services/           ← Business logic (pure functions, no HTTP)
│   │   ├── html_parser.py  ← AWR HTML → AWRData (BeautifulSoup4 + lxml)
│   │   ├── comparator.py   ← Period A vs Period B delta engine
│   │   ├── health_scorer.py ← 0-100 weighted health score
│   │   ├── rca_engine.py   ← Rule-based root-cause analysis
│   │   ├── comparator.py   ← Period A vs B delta + workload pattern detection
│   │   ├── mock_data.py    ← Synthetic data for demo/dev mode
│   │   ├── advanced_analytics.py ← Single-period analytics (cursor health, parse ratios)
│   │   ├── awr_intelligence.py ← AI narrative + intelligence pipeline
│   │   ├── data_source.py  ← Shared in-memory store (resolve_period_data)
│   │   ├── dot_connector.py ← Cross-metric dot-connector insights
│   │   ├── recommendations.py ← Recommendation generation logic
│   │   ├── oracle_pe_kb.py ← Oracle PE knowledge base
│   │   └── rag_narrative.py ← RAG-based narrative generation
│   ├── models/
│   │   └── snapshot.py (AWR snapshot models) + comparison.py (delta/regression models)
├── Pydantic v2 data models
├── templates/
│   └── index.html          ← Single-page frontend (~22,360 lines)
├── static/
│   └── (CSS/JS assets if any – CDN-served libs need no static files)
```

Design note: The frontend is a single HTML file containing all JavaScript and CSS. This keeps the distribution small and eliminates the need for a build step (no Webpack, Vite, or npm). Chart.js and Tailwind CSS are loaded from a public CDN at startup; they are not bundled.

5. End-to-End Request Flow

5.1 Application Startup

1. User double-clicks start.bat.
2. start.bat detects Python (tries py → python → python3).
3. start.bat kills any process already listening on port 8000.
4. pip installs/verifies all packages from requirements.txt.
5. _open_browser.py is launched in the background (start /b).
6. Uvicorn starts FastAPI on 127.0.0.1:8000.
7. _open_browser.py polls the socket every 0.5 s; once it connects it calls webbrowser.open('http://localhost:8000') and exits.
8. FastAPI's on_event('startup') fires an additional timer (1.5 s) as a fallback browser open.
9. Browser loads GET / → Jinja2 renders index.html → user sees the dashboard.

5.2 AWR File Upload & Parse Flow

1. User drags an AWR HTML file onto the upload zone in the browser.
2. JavaScript FileReader reads the file bytes client-side.
3. Fetch API posts multipart/form-data to POST /api/upload/awr with field period=baseline (or comparison).
4. FastAPI router (routers/upload.py) receives the UploadFile object.
5. services/html_parser.py: parse_awr_html() is called with the raw HTML bytes.
6. BeautifulSoup4 (lxml back-end) parses the HTML DOM.
7. Regex and table-scraping routines extract: DB Info, Snapshot header, Load Profile, Wait Events, SQL Statistics, Efficiency Statistics, and I/O metrics.
8. Results are assembled into an AWRData Pydantic model and validated.
9. AWRData is stored in an in-memory dict: store[period] = awr_data.
10. Router returns JSON {status:'ok', period:'baseline', db_name:..., snap_range:...}.
11. Browser receives the JSON and updates the sidebar card with DB name and snap range.

5.3 Comparison & RCA Flow

1. User uploads a second AWR file as period=comparison.

2. Browser calls GET /api/compare/mock (which actually uses uploaded data if present).
3. services/comparator.py: compare_periods(baseline, comparison) computes deltas for every metric (absolute and percentage).
4. services/health_scorer.py: calculate_health_score() scores both periods (0–100).
5. services/rca_engine.py: run_rca() applies deterministic rules to produce a ranked list of findings with severity (critical/warning/info) and remediation SQL.
6. services/comparator.py (pattern detection integrated): detect_workload_patterns() analyses wait-event vectors and load-profile trends to identify one of 7 pre-defined workload patterns.
7. Combined JSON response is returned to the browser.
8. JavaScript renderComparisonDashboard() populates all chart canvases, metric cards, SQL regression table, wait-event delta table, and the RCA panel.
9. renderPatternNotes() appends workload-pattern banners to the dashboard.
10. The Download RCA Report button becomes visible.

6. AWR Parsing Engine

The parsing engine (`services/html_parser.py`) is the most complex component. Oracle AWR HTML reports are not versioned XML — they are human-formatted HTML tables whose structure varies by Oracle release (11g, 12c, 19c, 21c) and by patch level. The parser uses a multi-strategy approach to handle this variability.

6.1 Parse Strategy

- Primary: find `<table>` elements by their preceding `<h2>` or `<a name='...'/>` anchor tag (each AWR section has a named anchor).
- Fallback: find tables by header row content — if the first `<th>` contains expected column names (e.g., 'Event', '% DB time'), use that table.
- Value extraction: strip HTML, remove commas and whitespace, cast to float with safe defaults (0.0 for missing values).
- Version detection: parse the 'DB Release' field from the DB Information table; adjusts column indices for 11g vs 19c differences.

6.2 Sections Extracted

AWR Section	Data Extracted	Key Fields
DB Information	Database identity	DB Name, Instance, Host, Release, CPUs, Memory GB
Snapshot Info	Time range	Begin/End Snap ID, Begin/End Time, Elapsed Minutes
Load Profile	Throughput metrics (per sec)	DB Time, DB CPU, Redo, Logical Reads, Physical Reads
Top 10 Wait Events	Wait-event breakdown	Event name, Waits, Time(s), % DB time, Avg wait ms
SQL Statistics	Top SQL by various dimensions	SQL ID, Plan hash, Elapsed time, Executions, Gets
Instance Efficiency	Cache and parse ratios	Buffer cache hit%, Library cache hit%, Soft parse%
I/O Statistics	Physical I/O per file	Reads, Writes, Read time, Write time (per datafile)

6.3 Pydantic Data Model

Parsed data is validated and typed by the `AWRData` Pydantic model (`models/snapshot.py` + `models/comparison.py`). Key nested models:

- `AWRData` — top-level container: `db_name`, `instance`, `host`, `release`, `cpus`, `memory_gb`, `begin_snap`, `end_snap`, `begin_time`, `end_time`, `elapsed_min`, `db_time_min`.
- `LoadProfileEntry` — per-metric row: `name` (str), `per_second` (float), `per_txn` (float).
- `WaitEvent` — event name (str), `waits` (int), `time_s` (float), `pct_db_time` (float), `avg_wait_ms` (float).
- `SqlStat` — `sql_id` (str), `plan_hash` (str), `elapsed_s` (float), `executions` (int), `elapsed_per_exec` (float), `cpu_pct` (float), `gets` (int).
- `EfficiencyStats` — `buffer_cache_hit_pct`, `library_cache_hit_pct`, `soft_parse_pct`, `latch_hit_pct` (all float).

7. Key Algorithms & Scoring

7.1 Average Active Sessions (AAS)

AAS is the fundamental Oracle saturation metric. It measures how many database sessions were active on average during the snapshot window.

$$\text{AAS} = \text{DB Time (seconds)} \div \text{Elapsed Time (seconds)}$$

When AAS exceeds the number of CPU cores, the database is CPU-saturated. OraVision computes AAS from the Load Profile section and compares it against the CPUs field from DB Information.

7.2 Bottleneck Classification

The comparator classifies the dominant bottleneck from the wait-event vector:

Bottleneck Type	Detection Rule	Primary Signal
CPU	CPU wait% $\geq$ 70% of total DB time	High 'DB CPU' row in Load Profile
I/O	Physical I/O waits $\geq$ 15% AND $>$ 30% of CPU wait%	db file sequential/scattered read events
Concurrency	Latch or lock waits $\geq$ 10% of total DB time	latch: ... / enq: TX events
None	No single category dominates	Balanced wait profile

7.3 Health Score (0–100)

The health scorer (services/health_scorer.py) starts at 100 and applies deductions based on threshold breaches. Final score maps to a risk band.

Metric	Good (no deduction)	Deduction	Max Deduction
Buffer cache hit%	$\geq$ 95%	$-(95 - \text{actual}) \times 1.0$	-25
Library cache hit%	$\geq$ 99%	$-(99 - \text{actual}) \times 2.0$	-10
Soft parse%	$\geq$ 95%	$-(95 - \text{actual}) \times 0.5$	-15
Latch hit%	$\geq$ 99%	$-(99 - \text{actual}) \times 3.0$	-15
Wait event spikes	$<$ 20% DB time each	-5 per event over threshold	-20
SQL elapsed per exec	$<$ 1 s	-5 if any SQL $>$ 10 s avg	-15

Score bands:

Score Range	Band	Colour	Interpretation
90–100	Excellent	Green	No significant issues
75–89	Good	Teal	Minor inefficiencies, monitor
60–74	Warning	Yellow	Performance degradation likely
40–59	Poor	Orange	Active tuning required
0–39	Critical	Red	Immediate intervention needed

7.4 SQL Regression Classification

Each SQL statement present in either period is classified by comparing its metrics:

Class	Criteria	Meaning
-------	----------	---------

new_offender	SQL absent in baseline, high elapsed in comparison	New problematic SQL introduced
regression	Elapsed/exec increased $\geq 30\%$ AND executions stable	Plan change or data-volume degradation
load_increase	Executions increased $\geq 50\%$, elapsed/exec stable	Same SQL called far more often
improved	Elapsed/exec decreased $\geq 20\%$	Performance gain — validate intentional
stable	Changes within $\pm 20\%$ across all metrics	No significant movement

7.5 Workload Pattern Detection

The pattern detector (services/comparator.py (pattern detection integrated)) inspects the wait-event vectors and load-profile deltas from both periods to identify one or more of 7 named patterns:

Pattern	Detection Signal	Typical Root Cause
RMAN Backup Storm	Significant increase in 'backup: ...' or 'RMAN ...' wait events	Unscheduled / overlapping RMAN job
Redo / Commit Storm	log file sync waits dominate, redo size spike in Load Profile	Batch DML, AUTOCOMMIT abuse
I/O Storm	db file scattered/sequential read spikes > 30% DB time	Full-table scans, missing indexes
Latch / Concurrency	latch: cache buffers chains, enq: TX waits spike	Hot blocks, row-level lock contention
Hard Parse Storm	parse count (hard) in Load Profile spikes, library cache misses	Missing bind variables, cursor sharing
Parallel Query Explosion	PX Deq waits, parallel sessions count spikes	Excessive DOP, parallel query flooding
Volume Explosion	Logical reads, physical reads, and redo all spike together	Unplanned data volume or batch job

8. API Reference

All endpoints are served under <http://127.0.0.1:8000>. FastAPI automatically generates interactive Swagger UI documentation at <http://127.0.0.1:8000/docs>.

Method	Path	Description	Returns
GET	/	Serve main dashboard HTML (Jinja2 render)	HTML
GET	/api	API metadata and endpoint directory	JSON
POST	/api/upload/awr	Upload & parse AWR HTML file. Field: label=<string identifier>, file=<upload>	JSON
GET	/api/snapshots/	List all loaded periods with DB name and snap range	JSON
GET	/api/compare/mock	Full comparison report (uses uploaded data or mock)	JSON
GET	/api/compare/health/{period}	Health score and check list for one period	JSON
GET	/api/sql/top/{period}	Top SQL statistics for a period	JSON
GET	/api/waits/{period}	Top 10 wait events for a period	JSON
GET	/api/recommendations/{period}	Ranked recommendations list for a period	JSON
GET	/api/overview/{period}	Single-call summary: DB info, KPIs, waits, SQL	JSON

8.1 Upload Request Format

```
POST /api/upload/awr
Content-Type: multipart/form-data
```

```
Fields:
  period  string  'any string identifier, e.g. 'good', 'bad', 'uploaded''
  file    binary  AWR HTML report (*.html or *.htm, up to 80 MB)
```

```
Response (200 OK):
{ "status": "ok", "upload_id": "good", "db_name": "ORCL",
  "begin_snap": 1234, "end_snap": 1238,
  "begin_time": "2026-04-01 02:00", "end_time": "2026-04-01 06:00" }
```

8.2 Comparison Response Structure

GET /api/compare/mock returns a large JSON object with these top-level keys:

- baseline — AWRData object for period 1
- comparison — AWRData object for period 2
- deltas — per-metric absolute and percentage deltas
- bottleneck — 'cpu' | 'io' | 'concurrency' | 'none'
- sql_regressions — list of SqlRegression objects (class, sql_id, delta fields)
- health_baseline / health_comparison — health score objects
- recommendations — ranked list of {severity, title, detail, sql} objects
- patterns — list of detected workload pattern names

9. Frontend Architecture

The entire frontend is a single HTML file: `backend/templates/index.html` (~22,360 lines). It is rendered once by Jinja2 and thereafter operates as a Single-Page Application (SPA) via JavaScript — no page reloads occur after initial load.

9.1 Page Structure

- Sidebar (fixed left, 64px collapsed / 256px expanded): upload zones for baseline and comparison files, period selector, navigation links, Download RCA Report button, Reset button.
- Main content area: tab-based sections — Dashboard Overview, SQL Analysis, Wait Events, Recommendations, Health Score.
- Modal overlays: error toasts, loading spinner.
- Chart canvases: Chart.js renders bar charts (wait events, load profile), doughnut charts (bottleneck breakdown), and line charts (AAS trend).

9.2 Key JavaScript Functions

Function	Responsibility
<code>uploadAWR(period)</code>	Reads file via <code>FileReader</code> , posts to <code>/api/upload/awr</code> , updates sidebar card
<code>loadComparison()</code>	Calls <code>/api/compare/mock</code> , hands data to <code>renderComparisonDashboard()</code>
<code>renderComparisonDashboard(d)</code>	Main render: populates all chart canvases, metric cards, delta tables
<code>detectWorkloadPatterns(...)</code>	Pure JS pattern detector: mirrors server-side logic for fast client recompute
<code>renderPatternNotes(patterns)</code>	Injects workload-pattern banner elements into the DOM
<code>renderHealthGauge(score)</code>	Renders the arc/gauge SVG for health score display
<code>renderSQLRegressions(rows)</code>	Populates the SQL regression table with colour-coded classification badges
<code>downloadReport()</code>	Assembles an HTML RCA report in-memory and triggers a download link
<code>resetAnalysis()</code>	Clears in-memory state, resets charts, hides Download button, resets UI
<code>switchTab(tab)</code>	Shows/hides content sections for the tab navigation

9.3 Data Flow Inside the Browser

```
Upload event
└─> FileReader.readAsArrayBuffer(file)
    └─> fetch POST /api/upload/awr
        └─> update sidebar card

Compare button click
└─> fetch GET /api/compare/mock
    └─> renderComparisonDashboard(data)
        ├──> Chart.js update (all canvases)
        ├──> renderSQLRegressions()
        ├──> setTimeout(120ms) → detectWorkloadPatterns()
        │                               └─> renderPatternNotes()
        └─> show Download RCA Report button
```

10. Deployment Options

Option	Audience	How to Start	Pros / Cons
start.bat (ZIP)	Non-technical Windows users	Double-click start.bat	Zero setup; auto-installs deps; Windows only
Manual Uvicorn	Developers / Linux / macOS	<code>cd backend && py -m uvicorn main:app --host 127.0.0.1 --port 8000</code>	Full control; any OS; no bat file needed
Docker (future)	Enterprise / shared team server	<code>docker run -p 8000:8000 oravision-awr-pro</code>	Isolated; sharable; requires Docker image build
Streamlit Cloud	Remote testers (browser only)	Deploy app.py to streamlit.io; share URL	No local install; public HTTPS; login gate included

10.1 start.bat Launch Sequence Detail

- Displays ASCII art banner and product description.
- Detects Python: tries py, python, python3 in that order — handles all Windows Python install styles.
- Kills any existing process on TCP port 8000 (netstat + taskkill) to avoid 'address in use' errors.
- Runs pip install -r backend\requirements.txt with trusted-host flags for corporate proxies.
- If pip fails: prints clear instructions (connect internet, or run pip manually) and pauses.
- Launches _open_browser.py as a background process (start /b).
- Starts Uvicorn. Server is now live.
- _open_browser.py detects the server (socket.create_connection retries 60×) and opens browser.
- After server stops (Ctrl+C): cmd /k keeps terminal open for inspection — no 'press any key' loss.

10.2 System Requirements

- OS: Windows 10/11 (start.bat); or any OS with Python 3.10+ (manual Uvicorn).
- Python: 3.10 or later, with pip. Must be on PATH.
- Internet: required for first-run pip install only. Subsequent runs are offline.
- Browser: any modern browser (Chrome 90+, Edge 90+, Firefox 90+). IE not supported.
- RAM: 256 MB minimum (typical AWR reports parse in < 100 ms, < 20 MB memory).
- Disk: ~80 MB for Python packages.
- Port: 8000 must be free (start.bat auto-clears it).

11. Security Considerations

Concern	Risk	Mitigation
AWR data sensitivity	AWR reports contain SQL text, execution plans, schema names	Server binds to 127.0.0.1 only. Data never leaves the machine. No logging of SQL content.
File upload	Malformed HTML could crash the parser	BeautifulSoup4 is resilient to malformed HTML. Parse errors return HTTP 422 with detail.
Port exposure	Port 8000 accessible to other processes on same host	Loopback-only binding (--host 127.0.0.1). Not reachable from network.
No authentication	Any user on the machine can access the dashboard	Acceptable for local single-user use. Streamlit deployment adds SHA-256 password gate.
CORS policy	CORS is restricted to localhost by default; configurable via CORS_ORIGINS environment variable	Restricted to localhost by default. Override with CORS_ORIGINS env var for shared-server deployments.
Dependency supply chain	pip packages could be compromised	Packages are pinned to minimum versions. Use --trusted-host only for PyPI (not arbitrary hosts).

12. Supported Oracle Versions

Oracle Version	Release Family	AWR Format Notes	Support Level
11g R2 (11.2.0.x)	11g	Older column layout; fewer wait-event metrics; no I/O Function stats	Full
12c R1 (12.1.0.x)	12c	Multitenant CDB/PDB annotations may appear; otherwise 11g-similar	Full
12c R2 (12.2.0.x)	12c	Additional Adaptive Plan metrics in SQL section	Full
18c (12.2 compat)	18c	Essentially 12.2 layout; annual release	Full
19c (long-term)	19c	Most common production version; parser tuned against 19c layout	Full
21c (innovation)	21c	Minor additional fields; backward compatible	Full
23c (free/cloud)	23c	Format largely compatible; may need minor tuning for new sections	Beta

Only HTML-format AWR reports are supported (generated by `awrrpt.sql` with output type `HTML`). Text-format AWR reports (`awrrpt.sql` type `TEXT`) and ADDM reports are not supported.

13. Dependency Manifest

13.1 Python (Backend) — requirements.txt

Package	Min Version	License	Purpose
fastapi	0.111.0	MIT	ASGI web framework; routing, dependency injection, OpenAPI
uvicorn[standard]	0.29.0	BSD-3	ASGI server; handles HTTP connections and keep-alive
python-multipart	0.0.9	Apache-2	Multipart form parsing for file uploads
jinja2	3.1.0	BSD-3	HTML template rendering for index.html
beautifulsoup4	4.12.0	MIT	HTML parsing of AWR reports
lxml	5.0.0	BSD-3	C-based XML/HTML back-end for BeautifulSoup4 (speed)
pydantic	2.0.0	MIT	Runtime data validation and serialisation
starlette	(bundled)	BSD-3	ASGI toolkit; CORS middleware, static files (bundled with FastAPI)

13.2 Frontend (CDN-loaded, no install needed)

Library	Version	CDN	Purpose
Tailwind CSS	3.x	jsDelivr / cdn.tailwindcss.com	Utility-first CSS framework, dark theme
Chart.js	4.x	jsDelivr	Canvas-based charting library
Heroicons	2.x	Inline SVG	UI icon set (no network request)

13.3 Runtime (not installed by pip)

Requirement	Version	How to get	Notes
Python	3.10+	python.org/downloads	Must be on PATH; tick 'Add Python to PATH' during install
pip	22+	Bundled with Python	start.bat calls py -m pip (not pip.exe) for reliability
Web browser	Any	Pre-installed	Chrome, Edge, or Firefox recommended

14. Glossary

AAS — Average Active Sessions. $\text{DB Time} \div \text{Elapsed Time}$. Fundamental Oracle saturation metric.

ASGI — Asynchronous Server Gateway Interface. Python async web server standard (successor to WSGI).

AWR — Automatic Workload Repository. Oracle's built-in performance snapshot facility (requires Diagnostics Pack licence on licensed editions).

awrrpt.sql — Oracle-supplied SQL script that generates an AWR HTML or text report from the data dictionary.

BeautifulSoup4 — Python library for parsing HTML and XML documents via a tree-based API.

CDN — Content Delivery Network. Libraries (Tailwind, Chart.js) are loaded from the internet at startup.

Chart.js — Open-source JavaScript charting library using HTML5 Canvas.

CDB/PDB — Container Database / Pluggable Database. Oracle multitenant architecture introduced in 12c.

DB Time — Total time spent by all foreground sessions (on-CPU + wait time). The primary Oracle workload metric.

DOP — Degree of Parallelism. Number of parallel execution servers used by a parallel query.

FastAPI — Modern Python web framework for building REST APIs with automatic OpenAPI/Swagger documentation.

Jinja2 — Python templating engine used to render index.html server-side before sending to browser.

lxml — Fast C-based Python XML/HTML parser library.

Pydantic — Python library for data validation using type annotations; version 2.x used here.

RCA — Root Cause Analysis. Systematic process of identifying the fundamental cause of a performance problem.

SQL ID — 13-character hash uniquely identifying a SQL statement's text in Oracle's shared pool.

Tailwind CSS — Utility-first CSS framework where styles are applied via short class names in HTML.

Uvicorn — ASGI server implementation; runs FastAPI applications in production.

Wait Event — An Oracle instrumentation point tracking why a session was not on CPU (e.g., waiting for disk I/O, a lock, or a latch).

